

PhishGuard AI: Final Defense Cheat Sheet & Panel Q&A

This document serves as your ultimate technical reference. It contains everything there is to know about the architecture of PhishGuard AI, including your cloud deployment, followed by a list of anticipated difficult questions the panel might ask, along with the perfect technical answers.

Part 1: The Master Technical Cheat Sheet

If you forget a technical detail during the presentation, refer to this section.

1. Deployment & Infrastructure

- **Cloud Hosting:** The Flask backend API is deployed on **Render.com** as a live, production-grade web service under the name “**PhishGuard AI**”.
- **Server Gateway:** The application is likely served using Gunicorn (as defined in your `Procfile`), which allows it to handle multiple concurrent requests from various users’ browser extensions.
- **Client Deployment:** The frontend is a local Chrome/Edge Browser Extension loaded via Developer Mode. It uses asynchronous `fetch()` API calls to securely transmit the payload to your Render.com endpoint.

2. Data Extraction (The Extension)

- **DOM Traversal:** The `background.js` script silently traverses the Document Object Model (DOM) of the webpage.
- **Payload Generation:** It extracts the `window.location.href` (URL), visible `innerText` (Text), `<img>` tags (Images), and `<audio>/<video>` tags. It packages this into a JSON payload and transmits it to the Flask server in real-time.

3. The 5 Machine Learning Modalities

1. URL Pipeline (`phishurl.py`)

- **Algorithm:** Ensemble `VotingClassifier` combining Random Forest, Gradient Boosting, and Logistic Regression.
- **Feature Engineering:** Extracts over 50 structural features (e.g., Shannon entropy, alphanumeric ratios, TLD maliciousness, path depth).
- **False Positive Prevention:** Hardcoded whitelist of over 500+ legitimate Indian government, banking, and institutional websites to ensure legitimate Indian sites aren’t accidentally flagged.

2. Text Pipeline (`phishtext.py`)

- **Algorithm:** Natural Language Processing (NLP).

- **Mechanics:** Generates **768-dimensional embeddings** to analyze semantics. It looks beyond keywords to understand the *context* of urgency (e.g., “Account suspended in 24 hours”) and social engineering.
3. **Image Pipeline (`phishimage.py`)**
 - **Algorithm:** Convolutional Neural Networks (CNNs) & Optical Character Recognition (OCR).
 - **Mechanics:** Generates **64-dimensional embeddings** to detect brand spoofing (e.g., a slightly altered SBI logo). Uses OCR to extract text embedded inside images, a common tactic attackers use to bypass text-based spam filters.
 4. **Audio Pipeline (`phishaudio.py`)**
 - **Mechanics:** Extracts audio waveforms to detect digital artifacts left behind by AI voice generators (deepfakes). Transcribes the audio to text to detect vishing (voice phishing) scripts.
 5. **Video Pipeline (`phishvideo.py`)**
 - **Mechanics:** Frame-by-frame extraction to detect visual manipulation (face-swapping) while simultaneously routing the video’s audio track through the audio pipeline.
4. **The Core Fusion Engine (`phishfusion.py`)**
 - **Base Weights:** URL (1.5), Text (1.2), Video (1.1), Image (1.0), Audio (1.0).
 - **Dynamic Weighting:** The base weight is multiplied by the model’s confidence score (how certain it is of its prediction).
 - **Embedding Correlation:** The engine takes the 768-dim Text vector and the 64-dim Image vector, truncates them to match dimensions, and calculates the **Cosine Similarity**.
 - **The Result:** If the models have high Cosine Similarity (they “agree” it’s an attack), it generates an **Agreement Multiplier** that boosts the final risk score. The final score is constrained between 0.01 and 0.99.
-

Part 2: Anticipated Panel Questions & How to Answer Them

The panel will try to test your understanding of the underlying technology. Memorize the concepts behind these answers.

Q1: “Why did you use a Fusion Engine instead of just taking the average of the 5 models?”

Who should answer: Mallikarjun **Answer:** “A simple average is dangerous because some modalities are inherently more predictive than others, and attackers often mix safe and malicious elements. For example, a phishing site might

have a perfectly safe-looking image, but a highly malicious URL and text. A simple average would dilute the risk score and let the attack through. Our Fusion Engine uses **Dynamic Weighting** (URL gets a 1.5 weight, Image gets 1.0) and **Cosine Similarity** to compare the multi-dimensional embeddings of the models. If the text and URL models strongly correlate, our Agreement Multiplier boosts the final risk score, ensuring the threat is blocked.”

Q2: “How does your system handle the fact that Text models output 768 dimensions but Image models output 64 dimensions during fusion?”

Who should answer: Mallikarjun or Viharika **Answer:** “To compute Cosine Similarity for the Agreement Multiplier, the vectors must be the same size. In our `_align_embeddings` function, we truncate the larger vector to match the topological space of the smaller vector (64 dimensions) before calculating the distance. While some high-level text nuance is lost in truncation, the core directional alignment remains mathematically sound enough to determine if the models agree on the threat vector.”

Q3: “How does your URL model avoid flagging legitimate Indian bank or government websites that might look outdated or have weird URLs?”

Who should answer: Navjyoth **Answer:** “False positives are a major issue in phishing detection. To solve this, during the feature engineering phase in our `phishurl` module, we integrated a massive validation list containing over 500 verified Indian institutional, banking, and government domains. If the domain structurally matches this legitimate list, our algorithm applies a massive legitimacy penalty to the final risk calculation, ensuring critical Indian infrastructure is never accidentally blocked.”

Q4: “Why use an Ensemble Voting Classifier for URLs instead of a Neural Network?”

Who should answer: Navjyoth **Answer:** “We chose a Voting Classifier combining Random Forest, Gradient Boosting, and Logistic Regression because URL analysis relies heavily on tabular, structural features—like the exact count of special characters, path depth, and entropy. Tree-based ensembles like Random Forest and Gradient Boosting consistently outperform deep neural networks on tabular data, require less computational overhead, and are much less prone to overfitting on URL datasets.”

Q5: “How does the image module detect text hidden inside an image?”

Who should answer: Viharika **Answer:** “Attackers often embed their phishing text (‘Click here to reset password’) directly into an image file because

traditional text-scrappers can't read it. To combat this, our **phishimage** module utilizes Optical Character Recognition (OCR). The OCR pipeline scans the image, extracts any rendered characters into a string, and we then pass that extracted string back into our NLP text pipeline to analyze for semantic urgency."

Q6: "Your backend is deployed on Render.com. How does the browser extension communicate with it without slowing down the user's browsing experience?"

Who should answer: Pranav or Mallikarjun **Answer:** "Performance was a priority. Our browser extension uses asynchronous JavaScript promises (`fetch` API) to send the payload to our Render.com backend. Because the ML processing happens entirely on the cloud server and not on the user's local machine, it doesn't consume the user's RAM or CPU. Furthermore, the extension operates silently in the background, only interrupting the user interface if the backend returns a high-risk confidence score."

Q7: "What if a page only has Text and a URL, but no Video or Audio? Does the system break?"

Who should answer: Mallikarjun **Answer:** "No, the architecture is highly modular and fault-tolerant. The extension only sends the payloads it finds. If there is no audio or video tag on the DOM, those specific pipelines simply do not execute. The Fusion Engine's dynamic weighting mathematically adjusts the denominator based *only* on the modalities that were successfully analyzed, ensuring the final score remains accurate."